

Sprint Raporu — Truth & Trust v1 / Foundation Day Sprint

Bu sprintte iki şey üzerine çalıştım: **lead analizinin doğruluğu** ve **sistemin parçalarının birbirleriyle düzgün konuşması**.

Sprint'in Çıkış Noktası

Aslında en baştaki amacım yeni bir **SDR ekranı (lead detail v2)** yapmaktı, tamamen gerçek SDR akışına uygun olacak şekilde. Bu kararın arkasındaki sebep şu: Çınar bey'le birlikte SDR framework'lerini araştırırken fark ettim ki **buy trigger, zamanlama, niyet sinyalleri gibi lokal SMB ölçeğinde satışa götüren tüm sinyalleri platformumuz zaten üretiyor** — ama dağınık şekilde, 5 ayrı tab'a dağılmış halde veriyor.

Bunun üzerine SDR v2 ekranı için **AI-native bir yaklaşım** planladım. Amaç şuydu: sistemin bir lead hakkında sahip olabileceği tüm veriyi SDR araştırma framework'lerine sokup, kullanıcıya sadece sonucu göstermek. Yani SDR'ın tab'lar arasında dolaşıp kendi kafasında bağlantı kurmasını engellemek — tüm açılar hazır önüne koymak. Yorum okumak, website analizine bakmak gerekmedikçe gerekmesin.

Ama bu hedefe gitmek için önce alttaki temeli sağlamlaştırmam gerekiyordu. Çünkü **AI ne kadar güzel bir ekran çizerse çizsin, arka tarafta uydurma üretiyorsa SDR güvenmez**. O yüzden bu sprintin asıl hedefi şu oldu:

AI'nın lead'ler hakkında attığı her lafın arkasında bir kanıt olsun, uyduramasın.

Spoiler: Sprint'in sonunda v2 ekranı projesini iptal ettim. Konuyu yeterince araştırın, sahiplenin kimse yok ekipte — o yüzden mevcut **v1'i olabildiğince SDR-first hale getirmeye** çalıştım, devamını da bir sonraki sprintte yapacağım. Ama altyapı taraf —yani bu sprintin asıl işi— tamamen tamam.

Sprint Kick-off (14 Mayıs — Foundation Day)

İlk gün literally yer döşedim ve takımın geri kalanını da hizaladım:

- **Sprint development plan'i** yazdım ([truth_trust_master_pipeline_dec5071b.plan.md](#)) — 7 ana faz, tüm track'ler tanımlı
- **ClickUp kurulumu** yapıldı, sprint task'ları subtask'larıyla beraber girildi
- **21 Mayıs sprint toplantısı** ayarlandı, 20 Mayıs son kontrol günü
- Takımın sprint sonu **deliverable'ları** yazıldı:
 - **Kaan**: Keyword araştırması + ICP/keyword bazlı landing page kurgusu + waitlist funnel + newsletter altyapısı
 - **Çınar**: SDR pain point araştırması + gerçek SDR'larla doğrulama + pain ranking + rakip analizi
- Çınar için **"Product Intelligence Engine"** rolü tanımlandı — fikir üreten değil, market sinyalini ürüne çeviren filtre

Kontrat dosyalarını da bu gün açtım ([src/lib/sdr-brain/contracts/](#)) — locale-output, nba-types, pain-point, severity, switch-signal, website-verification. Yani her worker'ın "ben şunu üretirim, sen de bunu okursun" diyebileceği ortak tipler artık var. [scripts/check-contracts.ts](#) adında bir CI script'i de ekledim — konratların bozulup bozulmadığını otomatik kontrol ediyor.

Önceden Nasıldı

AI worker'larımız (opener-writer, intelligence brief, objection predictor, NBA vs.) kafalarına göre çalışıyordu. Her biri Gemini'ye prompt atıyor, dönen text'i alıyor, ekrana basıyordu. Arada:

- **Ortak bir "kontrat" yoktu** — her worker farklı output veriyordu. Birinin severity: "high" dediğine başkası level: 3 diyordu.
- **Kanıt zorunluluğu yoktu** — AI "bu restoran büyüyor" diyebiliyordu, kimse "kaynağın ne?" diye sormuyordu. Hallucination riski tavandaydı.
- **Website doğrulama zayıftı** — Google'dan ilk gelen domain'i o lead'in sitesi sanıyorduk. Bazen alakasız bir site çıkıyor, AI o alakasız siteden brief ürettiyordu. Çöp girip çöp çıkıyordu.
- **Locale dağınıktı** — Türk lead'e bazen İngilizce mesaj gidiyordu, koda bakınca "burada mı kontrol ediyorduk, orada mı?" belli değildi.
- **Karar mantığı sezgiseldi** — "bu lead şimdi alır mı?" sorusunun cevabı bir-iki if'le veriliyordu, kayıt altında değildi.
- **Mobil lead detay yarım kalmıştı** — masaüstünde güzeldi ama telefonda gesture'lar, sticky CTA, voice note butonu hep yamuk duruyordu.
- **Gözümüz kapalıydı** — bir worker fail olduğunda haberimiz olmuyordu. Production'da neyin nasıl gittiğine "umuyoruz iyidir" diyorduk.

Mimari olarak özetlersem: agent-workers/ klasöründe her worker bağımsız bir adaydı, aralarında konuşma yoktu, ortak tipler yoktu, ortak gate yoktu.

Şimdi Nasıl

Üstüne bir **"Truth Layer"** kurdum. Worker'lar hala yerinde duruyor, ama artık aralarında bir hakemli sistem var.

1. Ortak Dil Katmanı — sdr-brain/contracts/

Yeni bir klasör açtım. İçinde [severity.ts](#), [pain-point.ts](#), [nba-types.ts](#), [locale-output.ts](#), [website-verification.ts](#) gibi tip sözleşmeleri var. Her worker bu tiplere uymak zorunda. Yani severity'nin nasıl bir şey olduğu artık **tek bir yerde** tanımlı. Önceden herkes kafasına göreymiş, şimdi standart var.

2. Gate'ler — sdr-brain/

[buying-readiness.ts](#), [avoidance-validator.ts](#), [decision-gates](#). Bunlar bekçi gibi davranıyor. Worker bir output ürettiğinde ekrana çıkmadan önce gate'e uğruyor:

- "Bu claim'in kanıtı var mı?" → yoksa düşür
- "Bu opener yasaklı kelime içeriyor mu?" → varsa düşür
- "Website doğrulandı mı?" → doğrulanmadıysa brief üretmeyi bile başlatma

Önceden output direkt user'a gidiyordu, şimdi arada filtre var.

3. Website Multi-Verify — agent-workers/website-multi-verify.ts

Eskiden tek kaynak vardı (Google'dan domain bul, bitti). Şimdi birden fazla kaynağa soruyor:

- **Bing brand search** ([apify/bing-brand-search.ts](#))

- **Companies House** ([apify/companies-house.ts](#) — UK firmaları için resmi kayıt)
- **Instagram bio scrape** ([apify/instagram-bio-scrape.ts](#))

Üçü de aynı domain'i işaret ediyorsa "tamam, doğru website" diyor. Aksi halde "emin değilim" deyip pipeline'ı durduruyor. Çoklu kaynak doğrulaması geldi.

4. Evidence-Bound Claims — [lead-detail-v2/ClaimWithEvidence.tsx](#)

UI tarafında yeni bir kural: her claim'in yanında **kaynak chip'i olmak zorunda**. Component bunu zorunlu prop yaptı. Yani frontend bile "kanıtsız claim render etmem" diyor. Worker tarafında da aynı kural — kanıt yoksa o cümle brief'e girmiyor. [lead-intelligence-brief.ts](#) bu yüzden 1062 satır büyüdü.

Ek olarak **AI dossier artık tıklanabilir** — sistem "yorumlar yüksek, şu konularda öne çıkıyor" derken o yorumlar highlight oluyor, kullanıcı tıklayıp gerçek yorumu okuyabiliyor. Yani kanıt sadece chip olarak değil, **görüntülenebilir** halde duruyor.

5. Locale Tek Noktadan — [lib/locale/lead-locale.ts](#)

Eskiden 3-4 yerde "lead'in dili ne?" sorusu soruluyordu. Şimdi tek bir helper var, herkes oraya soruyor: opener-writer, objection-predictor, intelligence-brief — hepsi `getLeadLocale(lead)` çağırıyor. Tek doğruluk kaynağı.

6. Telemetry & Dashboard — [lib/lead-detail/telemetry.ts](#) + [dashboards/truth-layer-v1.json](#)

Her worker artık ne yaptığını telemetry catalog'a yazıyor. Sentry'de hangi olayı izleyeceğimiz [sentry/truth-layer-alerts.yaml](#)'da tanımlı, dashboard config'i de dashboards/'a girdi. Gözümüz açıldı — bir worker yavaşladığında, fail ettiğinde, hallucinate ettiğinde alarm çalıyor.

Genel Tablo

Büyük bir master commit attım (Truth & Trust v1 Master Pipeline — Build Complete), öncesinde ve sonrasında bir sürü ufak fix + polish geçti. Sayılarla:

- **72 dosya değişti, ~13.000 satır eklendi**
- Yeni testler: lead-detail, icp-fit, sdr-brain, locale, agent-workers
- 3 yeni Apify worker (Bing brand search, Companies House, Instagram bio scrape)
- Yeni dashboard + Sentry alert config
- v2 lead detail mobile flow + telemetry catalog
- Smoke test'i yaptım, 20 lead çektim, bug'lar giderildi, Kaan'a test için sorunsuz teslim edildi
- Deployment'ı engelleyen güvenlik hususları araştırıldı ve çoğu kapatıldı (Sprint 1-6 PR serisi: webhook trust, SSRF guard, multi-tenant scope, IDOR fix, vs.)

Sprint Hedefleri + Başarı Kriterleri

Neyi neden yaptığımı dokümente ettim. Yani sadece kod değil, **karar günlüğü** de var artık. Yarın biri "bu niye böyle?" derse direkt dosyayı açıp okuyabilir.

Track B — Locale & Outreach Safety

Olay şu: AI bazen Türk lead'e İngilizce, İngiliz lead'e Türkçe mesaj yazıyordu. Artık lead'in nerede olduğuna + dilinin ne olduğuna bakıp opener-writer'ı doğru dile zorluyorum. [avoidance-validator.ts](#) da bu track'in parçası — yasaklı kelimeleri / yaklaşımları yakalayıp opener çıkmadan engelliyor.

Track A — Decision Quality

Eskiden lead'in "şimdi alır mı, almaz mı" kararı yarı sezgiseldi. Artık gate'ler var — buying readiness skoruna bakıyor, eşiği geçmediyse NBA (Next Best Action) farklı dallanıyor. Test coverage da var, deli saçması bir lead production'a kaçamıyor.

Track E — Website Verification

Lead'in websitesi gerçekten o lead'e mi ait, yoksa AI alakasız bir site mi kapmış? Şimdi çoklu doğrulama var — brand name, domain match, public signals, hepsine bakıyor. Doğrulanmamış websitelerden brief üretmeyi engelleyen bir gate var. 3 yeni Apify worker da bu track'i besliyor.

Track H — Observability & Quality

Bu en hoşuma giden kısımdı çünkü artık kör değiliz. Bir worker yavaşladığında / fail olduğunda / yanlış output ürettiğinde Sentry direkt bağıyor. Dashboard'da truth layer'ın canlı sağlığı görünüyor.

Track C — Evidence Calibration

Olay basit: AI bir şey söylediğinde kaynağını da gösterebilir. "Bu şirket büyüyor" diyemez tek başına — yanında "şu review'da, şu tarihte" kanıtı geliyor. Coverage testi bunu zorluyor, kanıt yoksa claim ekrana çıkmıyor. Dossier'daki claim chip'lerine tıklayınca gerçek kanıt açılıyor — review highlight oluyor, kullanıcı kaynağı doğrudan görüyor.

Track D — Truth-Grounded Intelligence Briefs

Sprintin en büyük tek dosya değişimi: [lead-intelligence-brief.ts](#) +1062 satır.

Briefler artık grounded — yani her cümlemin arkasında bir kanıt olmak zorunda, yoksa o cümle düşüyor. Test coverage da güzel:

- [lead-intelligence-brief-grounding.test.ts](#) (725 satır)
- [lead-intelligence-brief-website-gate.test.ts](#) (410 satır)

Website doğrulanmadıysa brief üretmiyor bile — sahte iddia üretme riski sıfır.

Track G + VK — Mobil Lead Detay v2

Lead detay v1 mobilde yamuktu — Track G ve VK kapsamında baştan elden geçti. Mobile gesture block, sticky CTA, voice note FAB — hepsi parmak ergonomisine göre yeniden ayarlandı. Playwright ile gerçek mobil viewport'ta e2e test yazıldı ([tests/e2e/lead-detail-mobile.spec.ts](#), 322 satır). Artık "açılıyor mu" değil "telefonda parmak doğru yere geliyor mu" testi var.

API Access (Kaan Entegrasyonu)

İptal oldu, Kaan bey görevlerini yapmadığı için.

Beta Tester & Niche Pack'ler (15–16 Mayıs)

Sprint altyapısı oturduktan sonra **gerçek lead'le test etme** zamanı geldi. İki beta tester aktif hale geldi, ikisinin de niche'i sisteme özel olarak açıldı:

Emirhan — Sürücü Kursu (15 Mayıs)

Emirhan'la 2 saat detaylı konuştum, **sürücü kursu nişini** seçti. Bunun üzerine:

- **Sürücü Kursu niche pack'i** açıldı: B / A2 / motor kursları tiplmesi, MEB onaylı sinyalleri, beyaz-mavi okul teması, otomatik kurs türü tespiti
- **Tek sayfalık zengin mockup şablonu**: hero + başarı oranı + süreç adımları + paket kartları + eğitimci profilleri + SSS + online randevu slotları + harita + iletişim formu + WhatsApp CTA
- Emirhan'a hesap açıldı, **20 sürücü kursu lead'i çekildi**, 5 tanesi gerçek satış fırsatı çıktı
- Emirhan sistemi gördü, beğendi, 1 saat konuştuk — tek isteği "üretilen mockup websitneyi daha da iyi yapabilir miyiz?"

Berkay — Kuyumcu (16 Mayıs)

Yeni beta tester Berkay'ın workspace'i hazırlandı, **kuyumcu nişi** Discovery'ye eklendi, kuyumcu-özel website mockup'ı geliştirildi. Berkay'a hesap açıldı, örnek mockup: leadacai.com/m/pxzdyic1qb

Bu Süreçte Üründe Yapılan Polish

- **Discovery sayfasına "lead sayısı seçici" geldi** (20 / 50 / 100 / 200 / 500) — artık discovery başlatırken kaç lead çekileceği user'ın elinde
- **Google Places** Türkiye aramalarında otomatik Türkçe diline geçiyor — Türk lead'lerinin metadata'sı artık doğru gelmeye başladı
- **Mockup pipeline reorder edildi** — artık scorer + audit + review **tamamlandıktan sonra** çalışıyor. Yani mockup, lead'in gerçek hakkındaki tüm bilgiyle üretildiği için çok daha zengin ve kişiselleştirilmiş çıkıyor

Yayın & Dağıtım (16 Mayıs)

Sprint task'ları bittikten sonra ürünü duyurmaya başladım:

- **Landing page yayınlandı** — ICP'ye uygun, AI-asistli versiyonu canlı
- **LinkedIn + Reddit postları** atıldı (ilk dağıtım denemesi)
- **Yeni ürün tanıtım videosu + ses kaydı** üretildi (30 saniyelik dikey)

Takım Operasyonu

Takım hızlanabilsin diye herkese kendi Notion altyapısını kurdum:

- **Çınar için Notion DB sistemi** — ürün özelliklerini sistemli planlayabilmesi için, "Features Rehberi" PDF'i ile beraber
- **Kaan için Notion template** — keyword + placement + landing page araştırmasını organize etsin diye, "SEO Rehberi" PDF'i ile beraber
- **Google Ads + landing page testing stratejisi** dokümanı yazıldı — bir sonraki sprintte ilk gerçek reklam test'lerini bu plan üzerinden yürüteceğiz

Toplamda Ne Oldu

Tek cümle özet: **AI'ın "uydurma" yapmasını engelledim, her iddianın kanıt zorunluluğu var.**

Yan ürünleri:

- 2 beta tester gerçek lead'le sistem üstünde
- Landing page canlı, dağıtım başladı
- Takımın geri kalanı kendi Notion altyapılarıyla paralel ilerleyebiliyor
- Lead detay v2 surface projesi iptal edildi, v1 SDR-first hale getirildi

Bir Sonraki Adım: AI-Native LeadAC

Bunları yaparken kafamda bir sonraki sprintlerin yönü netleşti ve **AI-Native LeadAC** whitepaper'ını yazmaya başladım (Çınar'la beraber bitireceğiz). Şu an sahip olduğumuz "her claim kanıta dayansın" temeli, aslında çok daha büyük bir şeyin alt katmanı: **LeadAC'i gerçek anlamda AI-native yapmak.**

AI-native demek, ürüne daha fazla AI feature eklemek demek değil; sistemin çalışma mantığını tamamen **intelligence-first** hale çevirmek demek. Şu an LeadAC çok gelişmiş bir **AI-assisted outbound platformu**: kullanıcı aksiyon alıyor, belirli worker'lar çalışıyor, sonuç üretiliyor. Yani AI şu an **araç** gibi davranıyor.

AI-native evrimde ise sistem araç olmaktan çıkıp **operasyonu yöneten bir varlık** gibi davranmaya başlıyor. Kullanıcı "Londra'daki premium sushi restoranlarından bu ay 15 demo çıkar" gibi bir hedef veriyor; sistem kendi ICP'sini genişletiyor, hangi subniche'in daha sıcak olduğunu analiz ediyor, geçmişte hangi outreach angle'lerinin işe yaradığını semantic memory'den hatırlıyor, hangi enrichment worker'larının çalışması gerektiğine karar veriyor, outreach'i optimize ediyor, başarısız pattern'leri bırakıp başarılı pattern'leri çoğaltıyor. Böylece LeadAC bir dashboard olmaktan çıkıp **yarı otonom bir outbound operating system'e** dönüşüyor.

Bunun temelinde birkaç kritik değişim var:

1. Static Chain → Dynamic Planning

Şu an lead_created → audit → scorer → dossier gibi önceden tanımlı akışlar var. AI-native sistemde planner runtime'da karar veriyor: bu lead hangi alt nişe ait, review yoğunluğu nasıl, website var mı, geçmişte benzer işletmelerde hangi angle cevap aldı, workspace'in sattığı offer ne, bütçe ve quota durumu nasıl? Buna göre execution DAG'i anlık oluşturuluyor. Yani sistem hardcoded workflow değil, **objective-driven reasoning** ile çalışıyor.

2. Worker Registry → Tool Graph

Şu an worker registry var ama AI-native modelde worker'lar **LLM tarafından çağrılabilen araçlara** dönüşüyor: `search_reviews()`, `generate_pitch()`, `deep_research()`, `score_opportunity()`, `build_sequence()`, `find_lookalikes()` gibi. Böylece Copilot sadece chat UI değil, gerçek bir operator haline geliyor. Sistem neyi ne zaman çalıştıracakını kendi seçiyor.

3. Semantic Memory: Retrieval → Behavioral Learning

Sistem sadece geçmiş bilgileri retrieve etmemeli; hangi opener'ın reply aldığı, hangi tone'un spam gibi algılandığı, hangi subniche'de hangi açının meeting açtığı gibi **pattern'leri öğrenmeli**. Zamanla şunu fark edebilmeli: *"London sushi restaurants respond better to reservation optimization angles than generic SEO messaging."* Bu bilgi sonraki generation'ları etkiler. Sistem prompt-driven değil, **outcome-driven** intelligence üretmeye başlar.

Gerçek **moat** burada oluşur: **proprietary outreach outcome graph**.

4. Niche Ontology — F&B Özelinde

Sistemin restoran ekonomisini anlaması gerekir: reservation flow, no-show problemi, table turnover, brunch economics, late-night traffic, delivery dependency, local SEO etkisi, booking provider friction, Instagram influence, review psychology gibi. **Fine dining ile coffee shop'un aynı davranmadığını bilmeli**. Fine dining prestige + reservation odaklı, bakery local discovery odaklı, ghost kitchen delivery conversion'a oynar, bar'lar late-night foot traffic'e. Bu farklar prompt engineering değil, **niche ontology**.

Bu sprintte sürücü kursu ve kuyumcu niche pack'lerini açarken aslında bu ontology'nin ilk taşlarını koymuş olduk — her niche'in kendi sinyalleri, kendi mockup formatı, kendi prompt'u var.

5. Agency Sales Intelligence

Hangi audit finding demo getirir, hangi pricing korkutur, hangi cold email spam gibi görünür, hangi phrasing güven verir, owner'lar hangi saatlerde daha responsive olur — bu SDR-level operational knowledge semantic layer'a işlenmeli. Yapı zamanla **playbook graph'e** dönüşür: *"No booking provider + high review count → booking optimization angle"* gibi rule'lar oluşur.

6. Reasoning Frameworks Katmanı

Uzun vadede sistemin içinde yalnızca bilgiler değil, **reasoning framework'ler** de bulunmalı: satış psikolojisi, persuasion modelleri, objection handling, positioning logic, cold outreach heuristics. Hormozi, Cialdini, Ogilvy, Challenger Sale, Predictable Revenue gibi kaynaklardan çıkarılmış operational heuristic'ler semantic graph'e dönüştürülebilir. Ama bu bir "master prompt" değil; **market knowledge + historical outcomes + sales psychology + retrieval memory + tool graph** birleşimi.

Sonuç

LeadAC'in evrimi şu: **AI kullanan bir SaaS olmaktan çıkıp, kendi kararlarını verebilen, geçmişten öğrenebilen, outreach stratejisini adapte eden, objective-driven çalışan bir outbound operating system'e dönüşmek.**

Bu sprintte attığımız "her claim kanıta dayansın" temeli olmasaydı bu vizyon hayal olurdu — çünkü kanıtsız bir sistem öğrenemez, **neyin işe yarayıp neyin yaramadığını** ayırt edemez. Önce truth layer'ı kurduk, şimdi üstüne intelligence layer'ı inşa edebiliriz.

Bir sonraki sprintte iki kola gideceğiz: bir taraftan iki beta tester'la (Emirhan + Berkay) gerçek lead verisini test etmeye devam, diğer taraftan AI-Native LeadAC whitepaper'ını bitirip **Faz 0'a** — yani planner'ı dinamik hale getirip worker'ları tool graph'e dönüştürme işine — başlayacağız.